ASSIGNMENT 2

Question 5: Generic Leaderboard

Create a generic Leaderboard<T> that stores entries and returns a sorted copy using a supplied Comparator. Teams are ranked by points descending, goal difference descending, then name ascending.

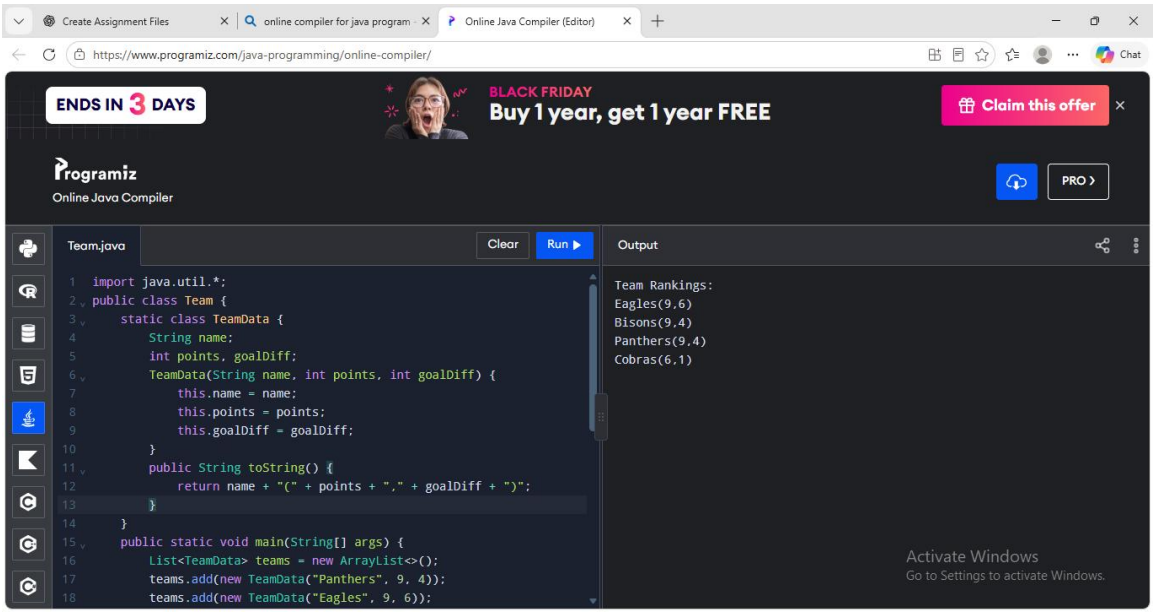
Java Program

```
import java.util.*;
class Team {
   final String name; final int points, goalDiff;
   Team(String n,int p,int g){name=n;points=p;goalDiff=g;}
   public String toString(){return
name+"("+points+","+goalDiff+")";}
}
class Leaderboard<T> {
   private final List<T> items=new ArrayList<>();
   private final Comparator<? super T> order;
   Leaderboard(Comparator<? super T> order){this.order=order;}
   void add(T item){items.add(item);}
   List<T> ranking(){List<T> copy=new
ArrayList<>(items);copy.sort(order);return copy;}
}
public class Task05_Leaderboard {
   public static void main(String[] args) {
      Comparator<Team>
byRank=Comparator.comparingInt((Team x)->x.points).reversed()
          .thenComparing(Comparator.comparingInt((Team x)-
>x.goalDiff).reversed())
```

```
            .thenComparing(x->x.name);
        Leaderboard<Team> board=new Leaderboard<>(byRank);
        board.add(new Team("Panthers",9,4)); board.add(new
Team("Eagles",9,6));
        board.add(new Team("Cobras",6,1)); board.add(new
Team("Bisons",9,4));
        System.out.println(board.ranking());
    }
}
```

Sample Output

[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]



Question 6: Priority Queue Triage

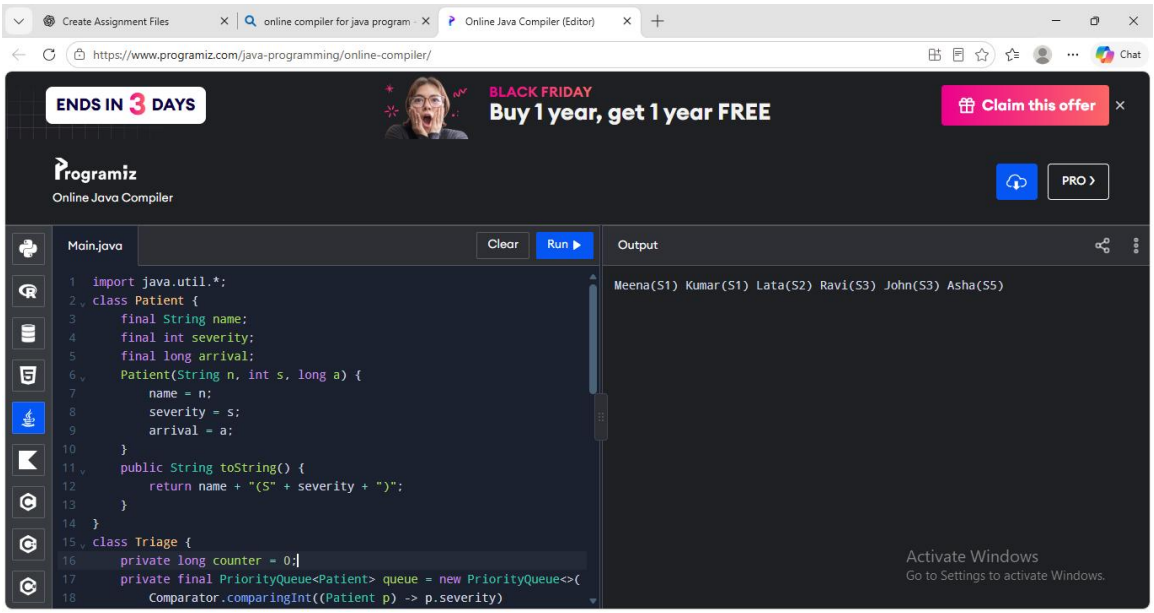
Use a PriorityQueue to serve patients by lower severity number first. Patients with equal severity are served in arrival order using an arrival counter.

Java Program

```
import java.util.*;
class Patient {
   final String name; final int severity; final long arrival;
   Patient(String n,int s,long a){name=n;severity=s;arrival=a;}
   public String toString(){return name+"(S"+severity+")";}
}
class Triage {
   private long counter=0;
   private final PriorityQueue<Patient> queue=new
PriorityQueue<>(
       Comparator.comparingInt((Patient p)-
>p.severity).thenComparingLong(p->p.arrival));
   void admit(String name,int severity){queue.add(new
Patient(name,severity,counter++));}
   Patient next(){return queue.poll();}
}
public class Task06_Triage {
   public static void main(String[] args) {
       Triage er=new Triage();
       er.admit("Ravi",3); er.admit("Meena",1); er.admit("John",3);
       er.admit("Asha",5); er.admit("Kumar",1); er.admit("Lata",2);
       StringBuilder order=new StringBuilder(); Patient p;
       while((p=er.next())!=null) order.append(p).append(' ');
       System.out.println(order.toString().trim());
   }
}
```

Sample Output

```
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```



Question 7: Collection Redesign

Use ArrayDeque for a playlist that needs efficient additions/removals at both ends. addFirst(), addLast(), and removeFirst() make the next-song operation clear and efficient.

Java Program

```
import java.util.*;
public class Task07_Redesign {
    // Example redesign based on Q2: use ArrayDeque for frequent additions/removals at both ends.
    public static void main(String[] args) {
        ArrayDeque<String> upNext=new ArrayDeque<>();
        upNext.addLast("Song A"); upNext.addLast("Song B");
        upNext.addFirst("Urgent Song");
        System.out.println("Next: "+upNext.removeFirst());
        System.out.println("Remaining: "+upNext);
        System.out.println("Design: ArrayDeque provides amortized
```

O(1) add/remove at both ends.");
    }
}

Sample Output

Next: Urgent Song
Remaining: [Song A, Song B]
Design: ArrayDeque provides amortized O(1) add/remove at both ends.

